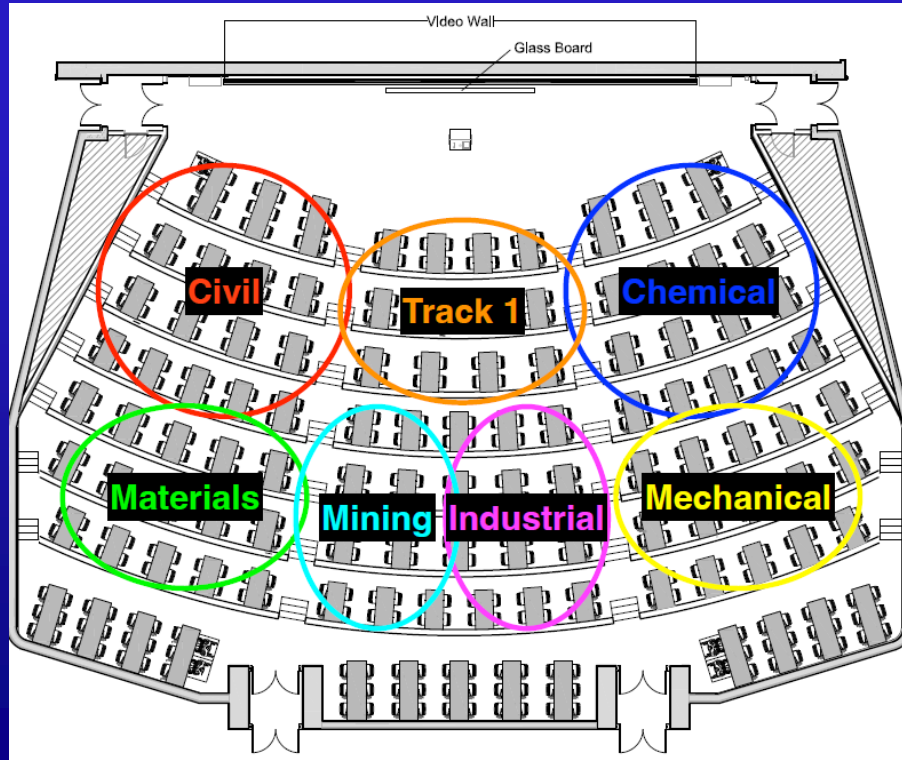


APS 106

FINAL REVIEW



RULES OF THE GAME

- Must use button on the table to answer question verbally and explain your answer – you will be called on to answer by one of the instructors
- Must tell us which discipline you're in before you answer for points (we'll rely on the honour system)
- If you get the answer right, you pick the next category
- If you get the answer wrong, the next person whose hand is up can steal
- Everyone is here to learn and review for the midterm so be kind to everyone who answers!

PANEL

CONTAINERS CONTINUED	READ, WRITE, MAYBE EVEN APPEND	CLASSY CLASSES
\$200	\$200	\$200
\$400	\$400	\$400
\$600	\$600	\$600
\$800	\$800	\$800
\$1000	\$1000	\$1000

FINAL JEOPARDY

CONTAINERS CONTINUED · \$200

What is the output?

```
andy = 'python'  
andy[0] = 'r'  
print(andy)
```

← BACK TO PANEL

[Question board](#)

CONTAINERS CONTINUED · \$200

What is the output?

```
andy = 'python'  
andy[0] = 'r'  
print(andy)
```

```
2 andy = 'python'  
----> 3 andy[0] = 'r'  
4 print(andy)
```

TypeError: 'str' object does not support item assignment

← BACK TO PANEL

[Question board](#)

CONTAINERS CONTINUED · \$400

What is the output?

```
count = {}  
count[(1, 2, 4)] = 5  
count[(4, 2, 1)] = 7  
count[(1, 2)] = 6  
count[(4, 2, 1)] = 2  
tot = 0  
  
for i in count:  
    tot += count[i]  
  
print(len(count) + tot)
```

- a) 25
- b) 17
- c) 16
- d) TypeError occurs as tuples can't be made keys of a dictionary

← BACK TO PANEL

[Question board](#)

CONTAINERS CONTINUED · \$400

What is the output?

```
count = {}  
count[(1, 2, 4)] = 5  
count[(4, 2, 1)] = 7  
count[(1, 2)] = 6  
count[(4, 2, 1)] = 2  
tot = 0  
  
for i in count:  
    tot += count[i]  
  
print(len(count) + tot)
```

c) 16

← BACK TO PANEL

[Question board](#)

CONTAINERS CONTINUED - \$600

What is the output?

```
def f1(x):  
    return x  
  
def f2(x):  
    return list(x)  
  
x = [1, 2, 3]  
  
print(x is f1(x))  
print(x is f2(x))  
print(x == f2(x))  
  
y = x  
x.append(24)  
print(x == y)
```

← BACK TO PANEL

[Question board](#)

CONTAINERS CONTINUED - \$600

What is the output?

```
def f1(x):  
    return x  
  
def f2(x):  
    return list(x)  
  
x = [1, 2, 3]  
  
print(x is f1(x))  
print(x is f2(x))  
print(x == f2(x))  
  
y = x  
x.append(24)  
print(x == y)
```

True
False
True
True

← BACK TO PANEL

[Question board](#)

CONTAINERS CONTINUED · \$800

What is the output?

```
phonebook = {'Akako' : 'Ke',
             'Ke': 'Freya',
             'Alya': 'Freya',
             'Freya': 'Frann'}

birthdays = {'September': ['Ke', 'Freya'],
             'March': ['Alya', 'Frann'],
             'July': ['Akako']}

names = set(phonebook.keys())
friends = set(phonebook.values())
check = names.intersection(friends)
print(check)

bday1 = set(birthdays['September'])[1]
bday2 = set(birthdays['March'])[1]
check = bday1.intersection(bday2)
print(check)
```

← BACK TO PANEL

[Question board](#)

CONTAINERS CONTINUED · \$800

What is the output?

```
phonebook = {'Akako': 'Ke',  
             'Ke': 'Freya',  
             'Alya': 'Freya',  
             'Freya': 'Frann'}  
  
birthdays = {'September': ['Ke', 'Freya'],  
             'March': ['Alya', 'Frann'],  
             'July': ['Akako']}  
  
names = set(phonebook.keys())  
friends = set(phonebook.values())  
check = names.intersection(friends)  
print(check)  
  
bday1 = set(birthdays['September'])[1]  
bday2 = set(birthdays['March'])[1]  
check = bday1.intersection(bday2)  
print(check)
```

```
{'Freya', 'Ke'}  
{'F', 'r', 'a'}
```

← BACK TO PANEL

[Question board](#)

CONTAINERS CONTINUED · \$1000

What is the output?

```
trade_inventory = {  
    'Emerald': 5,  
    'Gold': 10,  
    'Steel': 3,  
    'Diamond': 8  
}  
  
x = {}  
for a, b in trade_inventory.items():  
    x[b] = a  
  
print(x.pop(min(x)))  
x[max(x)] = "Silver"  
print(x)
```

← BACK TO PANEL

[Question board](#)

CONTAINERS CONTINUED · \$1000

What is the output?

```
trade_inventory = {  
    'Emerald': 5,  
    'Gold': 10,  
    'Steel': 3,  
    'Diamond': 8  
}  
  
x = {}  
for a, b in trade_inventory.items():  
    x[b] = a  
  
print(x.pop(min(x)))  
x[max(x)] = "Silver"  
print(x)
```

```
Steel  
{5: 'Emerald', 10: 'Silver', 8: 'Diamond'}
```

← BACK TO PANEL

[Question board](#)

READ, WRITE, MAYBE EVEN APPEND · \$200

What is written to file?

```
names = ['Charlie', 'Seb', 'Chris']
names.sort()
myfile = open("names.txt", "w")
for item in names:
    myfile.write(item + '\n')
myfile.close()
```

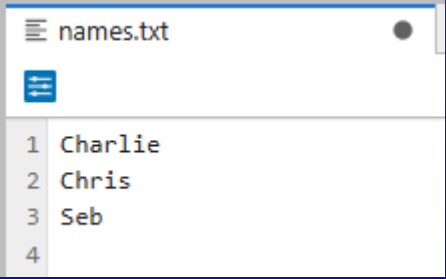
← BACK TO PANEL

[Question board](#)

READ, WRITE, MAYBE EVEN APPEND · \$200

What is written to file?

```
names = ['Charlie', 'Seb', 'Chris']
names.sort()
myfile = open("names.txt", "w")
for item in names:
    myfile.write(item + '\n')
myfile.close()
```

A screenshot of a text editor window titled 'names.txt'. The editor shows a list of names: 'Charlie', 'Chris', and 'Seb', each on a new line. The lines are numbered 1, 2, 3, and 4 on the left margin.

names.txt

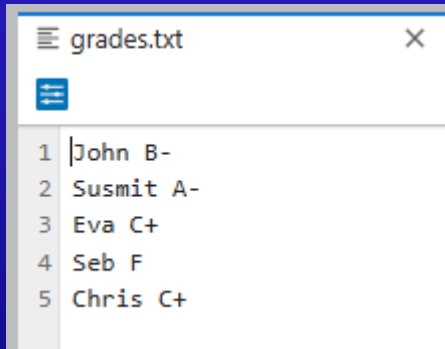
```
1 Charlie
2 Chris
3 Seb
4
```

← BACK TO PANEL

[Question board](#)

READ, WRITE, MAYBE EVEN APPEND · \$400

What is the output?



grades.txt

```
1 |John B-  
2 |Susmit A-  
3 |Eva C+  
4 |Seb F  
5 |Chris C+
```

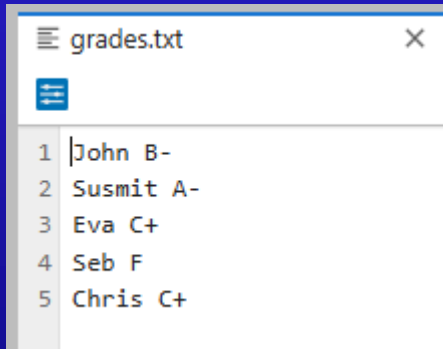
```
f = open("grades.txt", "r")  
for line in f.readlines():  
    print(line.strip())  
f.close()
```

← BACK TO PANEL

[Question board](#)

READ, WRITE, MAYBE EVEN APPEND · \$400

What is the output?



```
grades.txt
1 |John B-
2 |Susmit A-
3 |Eva C+
4 |Seb F
5 |Chris C+
```

```
f = open("grades.txt", "r")
for line in f.readlines():
    print(line.strip())
f.close()
```

```
John B-
Susmit A-
Eva C+
Seb F
Chris C+
```

← BACK TO PANEL

[Question board](#)

READ, WRITE, MAYBE EVEN APPEND · \$600

What is the output?

test.csv						
Delimiter: ,						
	APS111	04	20	2017	9:30	
1	APS111	04	20	2017	2:00	
2	APS164	04	18	2017	9:30	

```
import csv

with open('test.csv', newline = '') as csvfile:
    filereader = csv.reader(csvfile)
    for row in filereader:
        print(row[0] + ' exam on ' + row[2] + 'th at ' + row[-1])
```

← BACK TO PANEL

[Question board](#)

READ, WRITE, MAYBE EVEN APPEND · \$600

What is the output?

test.csv						
Delimiter: ,						
	APS111	04	20	2017	9:30	
1	APS111	04	20	2017	2:00	
2	APS164	04	18	2017	9:30	

```
import csv

with open('test.csv', newline = '') as csvfile:
    filereader = csv.reader(csvfile)
    for row in filereader:
        print(row[0] + ' exam on ' + row[2] + 'th at ' + row[-1])
```

```
APS111 exam on 20th at 9:30
APS111 exam on 20th at 2:00
APS164 exam on 18th at 9:30
```

← BACK TO PANEL

[Question board](#)

READ, WRITE, MAYBE EVEN APPEND · \$800

What is the output?

```
animals.txt  X  week13_lecture1_solutions_2X  +  
[Menu]  
1 there are 2 cats  
2 at least 1 dog  
3 parrots? probably like 24  
4 and another 18 cats?  
5 bunnies  
6 I sure hope there aren't more cats, I don't love cats
```

```
counts = {}  
  
with open('animals.txt', 'r') as animals:  
    for line in animals:  
        if "cat" in line:  
            if "cat" not in counts:  
                counts["cat"] = 3  
            else:  
                counts["cat"] += 1
```

← BACK TO PANEL

[Question board](#)

READ, WRITE, MAYBE EVEN APPEND · \$800

What is the output?

```
animals.txt  X  week13_lecture1_solutions_2X  +  
[Menu]  
1 there are 2 cats  
2 at least 1 dog  
3 parrots? probably like 24  
4 and another 18 cats?  
5 bunnies  
6 I sure hope there aren't more cats, I don't love cats
```

```
counts = {}  
  
with open('animals.txt', 'r') as animals:  
    for line in animals:  
        if "cat" in line:  
            if "cat" not in counts:  
                counts["cat"] = 3  
            else:  
                counts["cat"] += 1
```

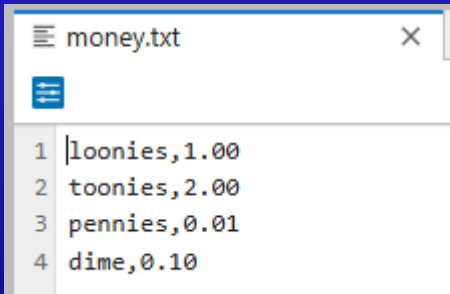
```
{'cat': 5}
```

← BACK TO PANEL

[Question board](#)

READ, WRITE, MAYBE EVEN APPEND · \$1000

What is the output?



```
money.txt
1 |loonies,1.00
2 |toonies,2.00
3 |pennies,0.01
4 |dime,0.10
```

```
dict1 = {}
with open("money.txt") as money_moves:
    line = money_moves.readline()
    while line != '':
        split = line.strip().split(',')
        dict1[split[0]] = split[1]
        line = money_moves.readline()

print(dict1)
```

← BACK TO PANEL

[Question board](#)

READ, WRITE, MAYBE EVEN APPEND · \$1000

What is the output?

```
money.txt
1 |loonies,1.00
2 |toonies,2.00
3 |pennies,0.01
4 |dime,0.10
```

```
dict1 = {}
with open("money.txt") as money_moves:
    line = money_moves.readline()
    while line != '':
        split = line.strip().split(',')
        dict1[split[0]] = split[1]
        line = money_moves.readline()

print(dict1)
```

```
{'loonies': '1.00', 'toonies': '2.00', 'pennies': '0.01', 'dime': '0.10'}
```

← BACK TO PANEL

[Question board](#)

CLASSY CLASSES - \$200

What is the output?

```
class Tester:
    def __init__(self, id = None) -> None:
        self.id = str(id)
        id = "224"

temp = Tester(123)
print(temp.id)
```

← BACK TO PANEL

[Question board](#)

CLASSY CLASSES - \$200

What is the output?

```
class Tester:  
    def __init__(self, id = None) -> None:  
        self.id = str(id)  
        id = "224"  
  
temp = Tester(123)  
print(temp.id)
```

123

← BACK TO PANEL

[Question board](#)

CLASSY CLASSES - \$400

What is the output?

```
class Journey:
    def __init__(self, origin, destination, duration):
        self.origin = origin
        self.destination = destination
        self.duration = duration

x = Journey('X', 'Y', 2.5)

print(isinstance(x, Journey))
print(isinstance(x, object))
print(isinstance(x.origin, str))
```

← BACK TO PANEL

[Question board](#)

CLASSY CLASSES - \$400

What is the output?

```
class Journey:
    def __init__(self, origin, destination, duration):
        self.origin = origin
        self.destination = destination
        self.duration = duration

x = Journey('X', 'Y', 2.5)

print(isinstance(x, Journey))
print(isinstance(x, object))
print(isinstance(x.origin, str))
```

True
True
True

← BACK TO PANEL

[Question board](#)

CLASSY CLASSES - \$600

What is the output?

```
class Animal:
    def __init__(self, name):
        self.weight = 16
        self.name = name
        self.age = 10

    def __str__(self):
        return self.name + " meows"

    def speak(self):
        print(self.name + " says hi", end = '')
        return 'skers'

cat = Animal("Whiskers")
print(cat.speak())
print(cat)
```

← BACK TO PANEL

[Question board](#)

CLASSY CLASSES - \$600

What is the output?

```
class Animal:
    def __init__(self, name):
        self.weight = 16
        self.name = name
        self.age = 10

    def __str__(self):
        return self.name + " meows"

    def speak(self):
        print(self.name + " says hi", end = '')
        return 'skers'

cat = Animal("Whiskers")
print(cat.speak())
print(cat)
```

```
Whiskers says hiskers
Whiskers meows
```

← BACK TO PANEL

[Question board](#)

CLASSY CLASSES - \$800

What is the output?

```
class StudentRecord:
    def __init__(self, name, midterm=0, project=0, final=0):
        self.name = name
        self.grades = [midterm, project, final]

    def remark(self, grade_component, number):
        self.grades[grade_component] += number

grade = 90
sr = StudentRecord("Ana Rao", grade, grade + 6, final = 90)
sr.remark(1, 3)
print(sum(sr.grades)/3)
```

← BACK TO PANEL

[Question board](#)

CLASSY CLASSES - \$800

What is the output?

```
class StudentRecord:
    def __init__(self, name, midterm=0, project=0, final=0):
        self.name = name
        self.grades = [midterm, project, final]

    def remark(self, grade_component, number):
        self.grades[grade_component] += number

grade = 90
sr = StudentRecord("Ana Rao", grade, grade + 6, final = 90)
sr.remark(1, 3)
print(sum(sr.grades)/3)
```

93.0

← BACK TO PANEL

[Question board](#)

CLASSY CLASSES · \$1000

What is the output?

```
class CityDweller:
    def __init__(self, name, gender, city):
        self.name = name
        self.gender = gender.lower()
        self.city = city.lower()
```

```
class CityCounter:
    def __init__(self, people):
        self.people = people

    def count_city(self, city_name):
        count = 0
        for person in self.people:
            if person.city == city_name.lower():
                count += 1
        return count
```

```
people = [
    CityDweller("Alice", "Female", "London"),
    CityDweller("Bob", "Male", "Paris"),
    CityDweller("Charlie", "Male", "London")
]

counter = CityCounter(people)
print("People from London:", counter.count_city("London"))
```

← BACK TO PANEL

[Question board](#)

CLASSY CLASSES · \$1000

What is the output?

```
class CityDweller:
    def __init__(self, name, gender, city):
        self.name = name
        self.gender = gender.lower()
        self.city = city.lower()
```

```
class CityCounter:
    def __init__(self, people):
        self.people = people

    def count_city(self, city_name):
        count = 0
        for person in self.people:
            if person.city == city_name.lower():
                count += 1
        return count
```

```
people = [
    CityDweller("Alice", "Female", "London"),
    CityDweller("Bob", "Male", "Paris"),
    CityDweller("Charlie", "Male", "London")
]

counter = CityCounter(people)
print("People from London:", counter.count_city("London"))
```

People from London: 2

← BACK TO PANEL

[Question board](#)

FINAL JEOPARDY



FINAL JEOPARDY

Write a class `LibraryEmployee` that has the following attributes:

- **name:** the name of the employee.
- **books:** a list of this employee's favourite `Books`.
- **num_coffees:** an integer representing how many coffees this employee has had today. This value should begin at zero when a `LibraryEmployee` is initialized.

The `LibraryEmployee` class should have an initializer that accepts the employee's **name** and **books** as arguments. These values should be assigned to the object's respective attributes.

Additionally, the `LibraryEmployee` class should have the following methods:

- **drink_coffee():** takes in an integer **num**, and increases this `LibraryEmployee`'s **num_coffees** attribute by that integer.
- **shush_hooligans():** takes in an integer **num_hooligans** and prints out "Shhh!" that many times. Also drinks a coffee once each time they shush a hooligan
- A **default method** that converts the `LibraryEmployee` into a string. It should return a string of all the employee's attributes formatted nicely for viewing (don't worry too much about parsing and printing the details from the `Books`).

BONUS: Also, write a class `Books` that has an initializer that sets the following attributes:

- **name:** the name of the book.
- **author:** the author of the book.
- **read_count:** the count of times this book has been read. This should be initialized to 0.

It should have a method **read()** that increments the **read_count** of that book and prints out something about the book's name.

← BACK TO PANEL

FINAL JEOPARDY - ANSWER

```
class LibraryEmployee:
    def __init__(self, name, books):
        self.name = name
        self.books = books
        self.num_coffees = 0

    def drink_coffee(self, num):
        self.num_coffees += num

    def shush_hooligans(self, num_hooligans):
        for i in range(num_hooligans):
            print("Shhh!")
            self.drink_coffee(1)

    def __str__(self):
        return f"Name: {self.name}\nFavourite Books: {self.books}\nNumber of Coffees: {self.num_coffees}"
```

```
class Book:
    def __init__(self, name, author):
        self.name = name
        self.author = author
        self.read_count = 0

    def read(self):
        self.read_count += 1
        print(f"{self.name} was a great book!")
```

BACK TO PANEL

```
TVHC = Book("The Very Hungry Caterpillar", "Eric Carle")
APS106 = Book("APS106 Textbook", "Sebastian Goodfellow")
HTBAH101 = Book("How to be a hooligan 101", "Chris Sutton")
```

```
charlie = LibraryEmployee("Charlie", [TVHC, APS106, HTBAH101])
charlie.drink_coffee(1)
charlie.shush_hooligans(3)
print(charlie)

charlie.books[0].read()
```